



Tech Solutions

SOFTWARE DESIGN DOCUMENT (SDD)

Swiftflitz Car Rental Management System

Document Reference: ORDAQ-SF-SDD-2026-001
Version: 1.0
Status: Final
Date: January 2026
Client: Swiftflitz Car Rental
Prepared By: Ordaq Development Team

CONFIDENTIAL

This document contains proprietary information intended solely for the use of Swiftflitz Car Rental and Ordaq Tech Solutions.

REVISION HISTORY

Version	Date	Author	Description
1.0	January 2026	Ordaq Dev Team	Initial release - Complete SDD for Swiftflitz system

TABLE OF CONTENTS

1. Introduction	4
1.1 Purpose	4
1.2 Scope	4
1.3 Definitions and Acronyms	4
1.4 References	5
2. System Overview	6
2.1 System Context	6
2.2 System Components	6
2.3 Design Constraints	7
3. Architecture Design	8
3.1 Architectural Pattern	8
3.2 System Architecture Diagram	8
3.3 Component Architecture	9
3.4 Technology Stack	10
4. Database Design	11
4.1 Database Selection	11
4.2 Entity-Relationship Model	11
4.3 Database Schema	12
4.4 Indexing Strategy	16
5. API Design	17
5.1 API Architecture	17
5.2 Authentication API	17
5.3 Vehicle API	18
5.4 Rental API	19
5.5 Customer API	21
5.6 Notification API	22
5.7 Error Response Format	22
6. Component Design	23
6.1 Public Website Components	23
6.2 Admin Panel Components	24
6.3 Service Layer Components	26
7. User Interface Design	28
7.1 Design Principles	28
7.2 Public Website Wireframes	28
7.3 Admin Panel Wireframes	29
7.4 Responsive Design	30
8. Security Design	31
8.1 Authentication & Authorization	31
8.2 Data Protection	32

8.3 Input Validation	33
8.4 Security Headers	33
9. Integration Design	34
9.1 WhatsApp Business API	34
9.2 Email Service (SMTP)	35
9.3 File Storage	36
10. Error Handling & Logging	37
10.1 Error Handling Strategy	37
10.2 Logging Architecture	38
11. Performance Design	39
11.1 Caching Strategy	39
11.2 Database Optimization	40
11.3 Asset Optimization	40
12. Deployment Architecture	41
12.1 Infrastructure Overview	41
12.2 Environment Configuration	42
12.3 CI/CD Pipeline	42
13. Appendices	43
A. Glossary	43
B. Database Schema SQL	44

1. INTRODUCTION

1.1 Purpose

This Software Design Document (SDD) provides a comprehensive technical blueprint for the Swiftflitz Car Rental Management System. It describes the architecture, component design, database structure, API specifications, and implementation details necessary for the development team to build the system.

The document serves as the primary technical reference throughout the development lifecycle and will be maintained as a living document to reflect design decisions and changes.

1.2 Scope

The Swiftflitz system consists of two integrated components:

- **Public Website: A customer-facing web application enabling vehicle browsing, booking, and quote requests**
 - Vehicle catalog with search and filtering
 - Online booking system with automated pricing
 - Quote request functionality for special vehicles
 - Responsive design for all devices
- **Admin Panel: An internal management dashboard for operations**
 - Vehicle fleet management
 - Rental lifecycle management
 - Customer database management
 - Reporting and analytics
 - Notification management

1.3 Definitions and Acronyms

Term	Definition
API	Application Programming Interface
CRUD	Create, Read, Update, Delete operations
JWT	JSON Web Token for authentication
MVC	Model-View-Controller architectural pattern
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPA	Single Page Application
UUID	Universally Unique Identifier
VPS	Virtual Private Server

1.4 References

- Software Requirements Document (SRD) - ORDAQ-SF-2026-001
- Ordaq Brand Guidelines - January 2026
- Laravel 11 Documentation - <https://laravel.com/docs>
- React 18 Documentation - <https://react.dev>
- WhatsApp Business API Documentation
- Paystack API Documentation (future integration)

2. SYSTEM OVERVIEW

2.1 System Context

The Swiftflitz Car Rental Management System operates as a web-based platform that connects car rental customers with the Swiftflitz business. The system interfaces with external services for notifications and future payment processing.

External Entity	Interface Type	Purpose
Customer	Web Browser (HTTPS)	Browse vehicles, make bookings, request quotes
Admin/Manager	Web Browser (HTTPS)	Manage fleet, rentals, customers, reports
WhatsApp Business API	REST API (HTTPS)	Send instant notifications to admin and customers
SMTP Email Server	SMTP/TLS	Send booking confirmations and system emails
AWS S3 (Optional)	REST API (HTTPS)	Store vehicle images and documents
Payment Gateway (Future)	REST API (HTTPS)	Process online payments

2.2 System Components

The system is organized into the following major components:

2.2.1 Presentation Layer

- Public Website (React/Blade)
 - Home page with featured vehicles
 - Vehicle catalog with advanced filtering
 - Booking and quote request forms
 - Static content pages
- Admin Panel (React SPA)
 - Dashboard with KPIs and charts
 - Management interfaces for all entities
 - Reporting and export functionality

2.2.2 Application Layer

- Laravel API Backend
 - RESTful API endpoints
 - Business logic services
 - Authentication and authorization
 - Request validation

2.2.3 Data Layer

- **MySQL/PostgreSQL Database**
 - Relational data storage
 - Transaction management
- **Redis Cache (Optional)**
 - Session storage
 - Query result caching
- **File Storage**
 - Local storage for development
 - AWS S3 for production

2.3 Design Constraints

The following constraints influence the system design:

Constraint	Description	Impact
Timeline	8-week development schedule	Prioritize core features, use proven technologies
Technology	Laravel + React/Blade stack	Architecture follows Laravel conventions
Hosting	VPS deployment	Design for single-server or small cluster
Budget	Cost-effective solutions	Use open-source tools, minimize external dependencies
Region	Ghana market focus	Support GHS currency, local business practices
Offline	Internet required	No offline capability in initial release

3. ARCHITECTURE DESIGN

3.1 Architectural Pattern

The system follows a Layered Architecture pattern combined with Domain-Driven Design principles. This approach provides clear separation of concerns, maintainability, and testability.

Layer	Responsibility	Components
Presentation	User interface rendering and interaction	React Components, Blade Views, CSS/Tailwind
Controller	HTTP request handling, input validation	Laravel Controllers, Form Requests
Service	Business logic implementation	Service Classes, Actions
Repository	Data access abstraction	Eloquent Repositories, Query Builders
Model	Data representation and relationships	Eloquent Models, DTOs
Infrastructure	External service integration	Mail, Notifications, Storage Drivers

3.2 System Architecture Diagram

The following table represents the system architecture layers and their interactions:

USERS	Customer (Web Browser) Admin/Manager (Web Browser)
FRONTEND	Public Website (React/Blade) Admin Panel (React SPA)
API	Laravel REST API Authentication (Sanctum) RBAC
SERVICES	Vehicle Rental Booking Pricing Customer Notification Report
DATA	MySQL/PostgreSQL Redis Cache File Storage (Local/S3)
EXTERNAL	WhatsApp Business API SMTP Email Server AWS S3

3.3 Component Architecture

3.3.1 Frontend Architecture

The frontend uses a component-based architecture with the following structure:

```

frontend/
├── public-website/
│   ├── components/
│   │   ├── layout/           # Header, Footer, Navigation
│   │   ├── vehicle/         # VehicleCard, VehicleGrid, VehicleDetail
│   │   ├── booking/         # BookingForm, DatePicker, Summary
│   │   ├── search/          # SearchBar, Filters, Results
│   │   └── common/          # Button, Input, Modal, Alert
│   ├── pages/
│   ├── hooks/
│   ├── services/
│   └── utils/
├── admin-panel/
│   ├── components/
│   │   ├── dashboard/      # KPICard, Charts, ActivityFeed
│   │   ├── vehicles/       # VehicleTable, VehicleForm
│   │   ├── rentals/        # RentalTable, RentalDetail, StatusBadge
│   │   ├── customers/      # CustomerTable, CustomerForm
│   │   └── shared/         # DataTable, FormFields, FileUpload
│   ├── pages/
│   ├── hooks/
│   └── context/
├── shared/
│   ├── types/
│   ├── constants/
│   └── api/

```

3.3.2 Backend Architecture

The Laravel backend follows a service-oriented structure:

```

app/
├── Http/
│   ├── Controllers/
│   │   ├── Api/
│   │   │   ├── AuthController.php
│   │   │   ├── VehicleController.php
│   │   │   ├── RentalController.php
│   │   │   ├── CustomerController.php
│   │   │   └── ReportController.php
│   │   └── Web/
│   ├── Requests/           # Form validation classes
│   ├── Resources/          # API resource transformers
│   └── Middleware/
├── Services/
│   ├── VehicleService.php
│   ├── RentalService.php
│   ├── BookingService.php
│   ├── PricingService.php
│   ├── NotificationService.php
│   └── ReportService.php
└── Models/

```

- Repositories/
- Events/
- Listeners/
- Notifications/
- Policies/

3.4 Technology Stack

The following technologies are selected for the system implementation:

3.4.1 Backend Technologies

Technology	Version	Purpose
PHP	8.2+	Server-side programming language
Laravel	11.x	PHP web application framework
Laravel Sanctum	4.x	API token authentication
Laravel Spatie Permission	6.x	Role-based access control
Laravel Media Library	11.x	File upload and management
Intervention Image	3.x	Image processing and optimization

3.4.2 Frontend Technologies

Technology	Version	Purpose
React	18.x	JavaScript UI library
TypeScript	5.x	Type-safe JavaScript
Tailwind CSS	3.x	Utility-first CSS framework
React Router	6.x	Client-side routing
TanStack Query	5.x	Server state management
React Hook Form	7.x	Form handling and validation
Axios	1.x	HTTP client
date-fns	3.x	Date manipulation

3.4.3 Database & Infrastructure

Technology	Version	Purpose
MySQL	8.0+	Primary relational database
PostgreSQL	15+ (alt)	Alternative database option
Redis	7.x	Caching and session storage
Nginx	1.24+	Web server and reverse proxy
Supervisor	4.x	Process management for queues
Ubuntu	22.04 LTS	Server operating system

4. DATABASE DESIGN

4.1 Database Selection

MySQL 8.0+ is the primary database for this system. PostgreSQL 15+ is a supported alternative. Both provide the necessary features for transactional integrity, JSON support, and full-text search.

Criteria	MySQL 8.0	PostgreSQL 15
Transaction Support	InnoDB with full ACID	Native ACID compliance
JSON Support	Native JSON type	JSONB with indexing
Full-Text Search	InnoDB FTS	Native FTS
Laravel Support	Excellent (default)	Excellent
Hosting Availability	Universal	Common

4.2 Entity-Relationship Model

The following diagram shows the relationships between core entities:

USERS id (PK) name email role_id (FK) is_active	ROLES id (PK) name display_name permissions[]	VEHICLES id (PK) make, model category_id (FK) status rates	CATEGORIES id (PK) name description is_active
CUSTOMERS id (PK) name, email phone license_number is_blacklisted	RENTALS id (PK) vehicle_id (FK) customer_id (FK) status costs	INSPECTIONS id (PK) rental_id (FK) type mileage, fuel photos[]	QUOTE_REQUESTS id (PK) vehicle_id (FK) contact info status

4.2.1 Relationship Summary

Entity A	Relationship	Entity B	Cardinality
User	belongs to	Role	Many-to-One
Role	has many	Permission	Many-to-Many
Vehicle	has many	Rental	One-to-Many
Vehicle	belongs to	Category	Many-to-One
Customer	has many	Rental	One-to-Many
User	manages	Rental	One-to-Many
Rental	has one	PickupInspection	One-to-One
Rental	has one	ReturnInspection	One-to-One

Vehicle	has many	VehicleImage	One-to-Many
Rental	triggers	Notification	One-to-Many

4.3 Database Schema

The following sections detail each table structure with columns, types, and constraints.

4.3.1 Users Table

Column	Type	Constraint	Description
id	BIGINT UNSIGNED	PRIMARY KEY	Auto-increment
uuid	CHAR(36)	UNIQUE	Public identifier
name	VARCHAR(255)	NOT NULL	Full name
email	VARCHAR(255)	UNIQUE	Login email
password	VARCHAR(255)	NOT NULL	Hashed password
phone	VARCHAR(20)	NULLABLE	Contact phone
role_id	BIGINT UNSIGNED	FOREIGN KEY	References roles.id
is_active	BOOLEAN	DEFAULT true	Account status
last_login_at	TIMESTAMP	NULLABLE	Last login time
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update

4.3.2 Roles Table

Column	Type	Constraint	Description
id	BIGINT UNSIGNED	PRIMARY KEY	Auto-increment
name	VARCHAR(100)	UNIQUE	Role name (admin, manager)
display_name	VARCHAR(255)	NOT NULL	Human-readable name
description	TEXT	NULLABLE	Role description
is_system	BOOLEAN	DEFAULT false	System role (non-deletable)
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update

4.3.3 Vehicles Table

Column	Type	Constraint	Description
id	BIGINT UNSIGNED	PRIMARY KEY	Auto-increment
uuid	CHAR(36)	UNIQUE	Public identifier
category_id	BIGINT UNSIGNED	FOREIGN KEY	References categories.id
make	VARCHAR(100)	NOT NULL	Manufacturer
model	VARCHAR(100)	NOT NULL	Model name
year	SMALLINT UNSIGNED	NOT NULL	Manufacturing year
license_plate	VARCHAR(20)	UNIQUE	Registration number
vin	VARCHAR(17)	UNIQUE, NULLABLE	Vehicle ID number

color	VARCHAR(50)	NOT NULL	Exterior color
seats	TINYINT UNSIGNED	NOT NULL	Seating capacity
transmission	ENUM	NOT NULL	automatic, manual
fuel_type	ENUM	NOT NULL	petrol, diesel, hybrid, electric
engine_size	VARCHAR(20)	NULLABLE	Engine displacement
mileage	INT UNSIGNED	DEFAULT 0	Current odometer
features	JSON	NULLABLE	Array of features
daily_rate	DECIMAL(10,2)	NULLABLE	Daily rental price
weekly_rate	DECIMAL(10,2)	NULLABLE	Weekly rental price
monthly_rate	DECIMAL(10,2)	NULLABLE	Monthly rental price
price_visible	BOOLEAN	DEFAULT true	Show price on website
status	ENUM	NOT NULL	available, rented, maintenance, retired
condition_notes	TEXT	NULLABLE	Vehicle condition
insurance_info	JSON	NULLABLE	Insurance details
is_featured	BOOLEAN	DEFAULT false	Featured on homepage
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update
deleted_at	TIMESTAMP	NULLABLE	Soft delete

4.3.4 Customers Table

Column	Type	Constraint	Description
id	BIGINT UNSIGNED	PRIMARY KEY	Auto-increment
uuid	CHAR(36)	UNIQUE	Public identifier
name	VARCHAR(255)	NOT NULL	Full name
email	VARCHAR(255)	UNIQUE	Email address
phone	VARCHAR(20)	NOT NULL	Primary phone
alt_phone	VARCHAR(20)	NULLABLE	Alternative phone
address	TEXT	NULLABLE	Physical address
license_number	VARCHAR(50)	NOT NULL	Driver license
license_expiry	DATE	NOT NULL	License expiry date
id_type	ENUM	NOT NULL	ghana_card, passport, voter_id
id_number	VARCHAR(50)	NOT NULL	ID document number
date_of_birth	DATE	NULLABLE	Birth date
emergency_contact	JSON	NULLABLE	Emergency contact info
notes	TEXT	NULLABLE	Internal notes
is_blacklisted	BOOLEAN	DEFAULT false	Blacklist status
blacklist_reason	TEXT	NULLABLE	Reason for blacklist
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update
deleted_at	TIMESTAMP	NULLABLE	Soft delete

4.3.5 Rentals Table

Column	Type	Constraint	Description
id	BIGINT UNSIGNED	PRIMARY KEY	Auto-increment
uuid	CHAR(36)	UNIQUE	Public identifier
reference	VARCHAR(20)	UNIQUE	Booking reference (SF-XXXXXX)
vehicle_id	BIGINT UNSIGNED	FOREIGN KEY	References vehicles.id
customer_id	BIGINT UNSIGNED	FOREIGN KEY	References customers.id
manager_id	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	References users.id
status	ENUM	NOT NULL	pending, rented, returned, completed, cancelled, overdue
pickup_date	DATE	NOT NULL	Scheduled pickup
pickup_time	TIME	NOT NULL	Pickup time
return_date	DATE	NOT NULL	Scheduled return

return_time	TIME	NOT NULL	Return time
actual_return_date	DATETIME	NULLABLE	Actual return timestamp
pickup_location	VARCHAR(255)	NULLABLE	Pickup address
dropoff_location	VARCHAR(255)	NULLABLE	Return address
is_airport_pickup	BOOLEAN	DEFAULT false	Airport pickup flag
base_cost	DECIMAL(10,2)	NOT NULL	Base rental cost
extras_cost	DECIMAL(10,2)	DEFAULT 0	Insurance, add-ons
deposit_amount	DECIMAL(10,2)	DEFAULT 0	Security deposit
additional_charges	DECIMAL(10,2)	DEFAULT 0	Late fees, damage, cleaning
discount_amount	DECIMAL(10,2)	DEFAULT 0	Applied discount
total_cost	DECIMAL(10,2)	NOT NULL	Total amount
payment_status	ENUM	NOT NULL	pending, partial, paid, refunded
amount_paid	DECIMAL(10,2)	DEFAULT 0	Amount received
customer_notes	TEXT	NULLABLE	Customer requests
admin_notes	TEXT	NULLABLE	Internal notes
source	ENUM	DEFAULT website	website, admin, phone
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update
deleted_at	TIMESTAMP	NULLABLE	Soft delete

4.3.6 Rental Inspections Table

Column	Type	Constraint	Description
id	BIGINT UNSIGNED	PRIMARY KEY	Auto-increment
rental_id	BIGINT UNSIGNED	FOREIGN KEY	References rentals.id
type	ENUM	NOT NULL	pickup, return
inspector_id	BIGINT UNSIGNED	FOREIGN KEY	References users.id
mileage	INT UNSIGNED	NOT NULL	Odometer reading
fuel_level	ENUM	NOT NULL	empty, quarter, half, three_quarter, full
exterior_condition	TEXT	NULLABLE	Exterior notes
interior_condition	TEXT	NULLABLE	Interior notes
damage_notes	TEXT	NULLABLE	Damage description
damage_cost	DECIMAL(10,2)	DEFAULT 0	Estimated repair cost
cleaning_required	BOOLEAN	DEFAULT false	Cleaning needed
cleaning_cost	DECIMAL(10,2)	DEFAULT 0	Cleaning fee
photos	JSON	NULLABLE	Array of photo paths
video_path	VARCHAR(255)	NULLABLE	Video file path
signature	TEXT	NULLABLE	Customer signature (base64)
inspected_at	DATETIME	NOT NULL	Inspection timestamp
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update

4.3.7 Quote Requests Table

Column	Type	Constraint	Description
id	BIGINT UNSIGNED	PRIMARY KEY	Auto-increment
uuid	CHAR(36)	UNIQUE	Public identifier
reference	VARCHAR(20)	UNIQUE	Quote reference (QR-XXXXXX)
vehicle_id	BIGINT UNSIGNED	FOREIGN KEY	References vehicles.id
name	VARCHAR(255)	NOT NULL	Customer name
email	VARCHAR(255)	NOT NULL	Customer email
phone	VARCHAR(20)	NOT NULL	Customer phone
pickup_date	DATE	NOT NULL	Requested pickup
return_date	DATE	NOT NULL	Requested return
message	TEXT	NULLABLE	Additional notes
status	ENUM	DEFAULT pending	pending, contacted, converted, declined
admin_notes	TEXT	NULLABLE	Internal notes

contacted_at	DATETIME	NULLABLE	First contact time
converted_rental_id	BIGINT UNSIGNED	FOREIGN KEY, NULLABLE	Resulting rental
created_at	TIMESTAMP	NOT NULL	Record creation
updated_at	TIMESTAMP	NOT NULL	Last update

4.4 Indexing Strategy

Strategic indexes improve query performance for common operations:

Table	Index Name	Columns	Purpose
vehicles	idx_vehicles_status	status	Filter by availability
vehicles	idx_vehicles_category	category_id	Category filtering
vehicles	idx_vehicles_featured	is_featured, status	Homepage query
rentals	idx_rentals_status	status	Status filtering
rentals	idx_rentals_dates	pickup_date, return_date	Date range queries
rentals	idx_rentals_vehicle	vehicle_id, status	Vehicle availability
rentals	idx_rentals_customer	customer_id	Customer history
customers	idx_customers_email	email	Login/lookup
customers	idx_customers_phone	phone	Phone search
quote_requests	idx_quotes_status	status, created_at	Pending quotes

4.4.1 Full-Text Search Indexes

Full-text indexes support vehicle search functionality:

```
-- MySQL Full-Text Index
ALTER TABLE vehicles ADD FULLTEXT INDEX ft_vehicles_search
    (make, model, color, condition_notes);

-- Usage in query
SELECT * FROM vehicles
WHERE MATCH(make, model, color) AGAINST("Toyota Camry" IN NATURAL LANGUAGE
MODE);
```

5. API DESIGN

5.1 API Architecture

The system exposes a RESTful API following industry best practices:

- Base URL: /api/v1
- Content-Type: application/json
- Authentication: Bearer token (Laravel Sanctum)
- Rate Limiting: 60 requests/minute (configurable)
- Versioning: URL-based (/api/v1/, /api/v2/)

5.1.1 HTTP Methods

Method	Usage	Example
GET	Retrieve resources	GET /api/v1/vehicles
POST	Create new resource	POST /api/v1/rentals
PUT	Full update	PUT /api/v1/vehicles/{id}
PATCH	Partial update	PATCH /api/v1/rentals/{id}/status
DELETE	Remove resource	DELETE /api/v1/vehicles/{id}

5.1.2 Response Format

All API responses follow a consistent structure:

```
{
  "success": true,
  "message": "Operation completed successfully",
  "data": { ... },
  "meta": {
    "current_page": 1,
    "per_page": 15,
    "total": 45,
    "last_page": 3
  }
}
```

5.2 Authentication API

POST /api/v1/auth/login

Authenticate user and retrieve access token

Request: { "email": "admin@swiftflitz.com", "password": "*****" }

Response: { "success": true, "data": { "token": "...", "user": {...}, "expires_at": "..." } }

POST /api/v1/auth/logout

Invalidate current access token

Request: Headers: Authorization: Bearer {token}

Response: { "success": true, "message": "Logged out successfully" }

GET /api/v1/auth/me

Get current authenticated user

Request: Headers: Authorization: Bearer {token}

Response: { "success": true, "data": { "id": 1, "name": "...", "role": {...} } }

5.3 Vehicle API

5.3.1 Public Endpoints

GET /api/v1/public/vehicles

List available vehicles with filtering

Request: Query: ?category=suv&transmission=automatic&min_price=100&max_price=500&page=1

Response: { "success": true, "data": [...], "meta": { "total": 25, ... } }

GET /api/v1/public/vehicles/{uuid}

Get single vehicle details

Request: Path: uuid = vehicle public identifier

Response: { "success": true, "data": { "uuid": "...", "make": "Toyota", ... } }

GET /api/v1/public/vehicles/{uuid}/availability

Check vehicle availability for dates

Request: Query: ?pickup_date=2026-02-01&return_date=2026-02-05

Response: { "success": true, "data": { "available": true, "price_breakdown": {...} } }

5.3.2 Admin Endpoints

GET /api/v1/admin/vehicles

List all vehicles (admin)

Request: Query: ?status=available&search=toyota&per_page=20

Response: { "success": true, "data": [...], "meta": {...} }

POST /api/v1/admin/vehicles

Create new vehicle

Request: { "make": "Toyota", "model": "Camry", "year": 2024, "category_id": 1, ... }

Response: { "success": true, "data": { "id": 15, "uuid": "...", ... } }

PUT /api/v1/admin/vehicles/{id}

Update vehicle

Request: { "daily_rate": 250.00, "status": "maintenance" }

Response: { "success": true, "data": { ... } }

DELETE /api/v1/admin/vehicles/{id}

Soft delete vehicle

Request: Path: id = vehicle ID

Response: { "success": true, "message": "Vehicle deleted" }

POST /api/v1/admin/vehicles/{id}/images

Upload vehicle images

Request: Content-Type: multipart/form-data, Body: images[] (files)

Response: { "success": true, "data": { "images": [...]} }

5.4 Rental API

5.4.1 Public Booking

POST /api/v1/public/bookings

Create new booking (customer)

Request: { "vehicle_uuid": "...", "pickup_date": "2026-02-01", "pickup_time": "09:00", "return_date": "2026-02-05", "return_time": "18:00", "customer": { "name": "...", "email": "...", "phone": "...", "license_number": "..." } }

Response: { "success": true, "data": { "reference": "SF-123456", "total_cost": 1000.00 } }

POST /api/v1/public/quote-requests

Submit quote request

Request: { "vehicle_uuid": "...", "name": "...", "email": "...", "phone": "...", "pickup_date": "...", "return_date": "...", "message": "..." }

Response: { "success": true, "data": { "reference": "QR-123456" } }

5.4.2 Admin Rental Management

GET /api/v1/admin/rentals

List all rentals

Request: Query: ?status=rented&from_date=2026-01-01&to_date=2026-01-31

Response: { "success": true, "data": [...], "meta": {...} }

POST /api/v1/admin/rentals

Create rental (admin)

Request: { "vehicle_id": 5, "customer_id": 10, "pickup_date": "...", ... }

Response: { "success": true, "data": { "id": 50, "reference": "SF-..." } }

PATCH /api/v1/admin/rentals/{id}/pickup

Process vehicle pickup

Request: { "mileage": 45000, "fuel_level": "full", "photos": [...], "notes": "..." }

Response: { "success": true, "data": { "status": "rented", ... } }

PATCH /api/v1/admin/rentals/{id}/return

Process vehicle return

Request: { "mileage": 45500, "fuel_level": "three_quarter", "damage_notes": "...", "photos": [...] }

Response: { "success": true, "data": { "status": "returned", "additional_charges": 50.00 } }

PATCH /api/v1/admin/rentals/{id}/complete

Approve and complete rental

Request: { "final_notes": "..." }

Response: { "success": true, "data": { "status": "completed" } }

GET /api/v1/admin/rentals/{id}/comparison

Get pickup vs return comparison

Request: Path: id = rental ID

Response: { "success": true, "data": { "mileage_diff": 500, "fuel_diff": "1 level", ... } }

5.5 Customer API

GET /api/v1/admin/customers

List all customers

Request: Query: ?search=john&is_blacklisted=false

Response: { "success": true, "data": [...] } }

POST /api/v1/admin/customers

Create customer

Request: { "name": "...", "email": "...", "phone": "...", "license_number": "...", ... }

Response: { "success": true, "data": { "id": 20, ... } }

GET /api/v1/admin/customers/{id}

Get customer details with rental history

Request: Path: id = customer ID

Response: { "success": true, "data": { "customer": {...}, "rentals": [...], "stats": {...} } }

PATCH /api/v1/admin/customers/{id}/blacklist

Toggle customer blacklist

Request: { "is_blacklisted": true, "reason": "Multiple damages" }

Response: { "success": true, "data": { "is_blacklisted": true } }

5.6 Notification API

POST /api/v1/admin/notifications/send

Send manual notification

Request: { "type": "whatsapp", "recipient": "+233...", "template": "booking_reminder", "data": {...} }

Response: { "success": true, "data": { "sent": true, "message_id": "..." } }

GET /api/v1/admin/notifications/logs

Get notification logs

Request: Query: ?type=whatsapp&from_date=2026-01-01

Response: { "success": true, "data": [...] } }

5.7 Error Response Format

All errors follow a consistent format with appropriate HTTP status codes:

```
// Validation Error (422)
{
  "success": false,
  "message": "Validation failed",
  "errors": {
    "email": ["The email field is required."],
    "phone": ["The phone format is invalid."]
  }
}

// Not Found (404)
{
```

```
"success": false,  
"message": "Vehicle not found"  
}  
  
// Unauthorized (401)  
{  
  "success": false,  
  "message": "Unauthenticated"  
}
```

6. COMPONENT DESIGN

6.1 Public Website Components

The public website is built with React components organized by feature:

6.1.1 Layout Components

Component	Description	Key Features
Header	Navigation bar with logo, menu, and CTA button	Fixed position, responsive collapse
Footer	Site links, contact info, newsletter signup	Consistent across all pages
PageWrapper	Layout container with header/footer	SEO meta tags, loading states
MobileMenu	Slide-out navigation for mobile	Animated, touch-friendly

6.1.2 Vehicle Components

Component	Description	Key Features
VehicleCard	Displays vehicle thumbnail, name, price, specs	Hover effects, quick actions
VehicleGrid	Responsive grid of VehicleCards	Pagination, infinite scroll option
VehicleDetail	Full vehicle page with gallery, specs, booking	Image carousel, tabs
VehicleGallery	Image gallery with lightbox	Zoom, swipe gestures
VehicleSpecs	Specification table/list	Icon-based display
PriceDisplay	Shows rates (daily/weekly/monthly)	Conditional "Request Quote"

6.1.3 Search & Filter Components

Component	Description	Key Features
SearchBar	Main search input with autocomplete	Debounce API calls
FilterPanel	Sidebar with filter options	Collapsible sections
FilterChip	Active filter indicator	Removable, clearable
DateRangePicker	Pickup/return date selection	Blocked dates, min duration
PriceRangeSlider	Price range filter	Dual handles, live update
SortDropdown	Sort options selector	Price, popularity, newest

6.1.4 Booking Components

Component	Description	Key Features
BookingForm	Multi-step booking form	Validation, progress indicator
CustomerForm	Customer details input	Required fields, formatting
BookingSummary	Order summary with pricing	Itemized breakdown
DateTimePicker	Combined date and time selection	Business hours validation
QuoteRequestForm	Quote request for non-priced vehicles	Simpler than booking

ConfirmationPage	Booking success display	Reference number, email sent
-------------------------	-------------------------	------------------------------

6.2 Admin Panel Components

6.2.1 Dashboard Components

Component	Description	Key Features
DashboardLayout	Admin layout with sidebar	Collapsible sidebar, breadcrumbs
KPICard	Single metric display	Value, trend, icon
KPIGrid	Grid of KPI cards	Responsive layout
RevenueChart	Line/bar chart for revenue	Date range selection
StatusChart	Pie/donut for rental status	Interactive segments
ActivityFeed	Recent activity timeline	Real-time updates
OverdueAlerts	Overdue rental warnings	Priority sorting, actions

6.2.2 Data Management Components

Component	Description	Key Features
DataTable	Reusable data table	Sorting, filtering, pagination
TableActions	Row action buttons	Edit, delete, view, custom
SearchInput	Table search with debounce	Multiple column search
BulkActions	Multi-select operations	Checkboxes, action dropdown
ExportButton	Export to PDF/Excel	Selected or all records
ColumnSelector	Show/hide columns	Saved preferences

6.2.3 Vehicle Management Components

Component	Description	Key Features
VehicleTable	Vehicle listing table	Status badges, quick edit
VehicleForm	Create/edit vehicle form	Multi-section, validation
ImageUploader	Drag-drop image upload	Preview, reorder, delete
PricingSection	Rate configuration	Daily/weekly/monthly inputs
FeatureSelector	Multi-select features	Checkboxes, custom add
MaintenanceLog	Maintenance history	Date, type, notes, cost

6.2.4 Rental Management Components

Component	Description	Key Features
RentalTable	Rental listing with status	Color-coded statuses
RentalDetail	Full rental view	Timeline, actions, documents
RentalForm	Create/edit rental	Customer/vehicle lookup
StatusBadge	Rental status indicator	Pending, Rented, Completed...
StatusTimeline	Visual status history	Timestamps, actor info

InspectionForm	Pickup/return inspection	Photos, damage, fuel, mileage
ComparisonView	Pickup vs return comparison	Side-by-side, differences
ChargesCalculator	Additional charges input	Auto-calculate totals

6.3 Service Layer Components

Backend services encapsulate business logic and are injected into controllers:

6.3.1 Core Services

```
class VehicleService {
    public function list(array $filters): LengthAwarePaginator;
    public function findByUuid(string $uuid): ?Vehicle;
    public function create(array $data): Vehicle;
    public function update(Vehicle $vehicle, array $data): Vehicle;
    public function delete(Vehicle $vehicle): bool;
    public function checkAvailability(Vehicle $vehicle, Carbon $start, Carbon
$end): bool;
    public function uploadImages(Vehicle $vehicle, array $files): Collection;
    public function setFeatured(Vehicle $vehicle, bool $featured): Vehicle;
}
```

```
class RentalService {
    public function create(array $data): Rental;
    public function processPickup(Rental $rental, array $inspection): Rental;
    public function processReturn(Rental $rental, array $inspection): Rental;
    public function complete(Rental $rental): Rental;
    public function cancel(Rental $rental, string $reason): Rental;
    public function calculateCharges(Rental $rental): array;
    public function getComparison(Rental $rental): array;
    public function generateReference(): string;
}
```

```
class PricingService {
    public function calculateRentalCost(
        Vehicle $vehicle,
        Carbon $pickupDate,
        Carbon $returnDate,
        array $extras = []
    ): PriceBreakdown;

    public function calculateLateFee(Rental $rental, Carbon $actualReturn):
float;
    public function applyDiscount(float $amount, string $code): float;
}
```

6.3.2 Notification Service

```
class NotificationService {
    public function sendBookingConfirmation(Rental $rental): void;
    public function sendQuoteRequestAlert(QuoteRequest $quote): void;
    public function sendReturnReminder(Rental $rental): void;
    public function sendOverdueAlert(Rental $rental): void;
    public function sendAdminAlert(string $type, array $data): void;

    // Channel-specific
    protected function sendWhatsApp(string $to, string $template, array $data):
bool;
    protected function sendEmail(string $to, Mailable $mailable): bool;
}
```

6.3.3 Report Service

```
class ReportService {
```

```
        public function getDashboardStats(Carbon $from, Carbon $to):  
DashboardStats;  
        public function getRevenueReport(Carbon $from, Carbon $to): RevenueReport;  
        public function getVehicleUtilization(Carbon $from, Carbon $to):  
Collection;  
        public function getCustomerReport(int $customerId): CustomerReport;  
        public function exportToPdf(string $report, array $data): string;  
        public function exportToExcel(string $report, array $data): string;  
    }
```

7. USER INTERFACE DESIGN

7.1 Design Principles

The UI design follows these core principles:

- **Clarity: Clean layouts with clear visual hierarchy**
 - Adequate whitespace between elements
 - Consistent typography scale
 - Obvious call-to-action buttons
- **Consistency: Uniform patterns across all interfaces**
 - Reusable component library
 - Consistent color usage
 - Standard interaction patterns
- **Efficiency: Minimize steps to complete tasks**
 - Smart defaults
 - Auto-complete and suggestions
 - Keyboard navigation support
- **Feedback: Clear system responses**
 - Loading indicators
 - Success/error messages
 - Form validation feedback

7.2 Public Website Wireframes

Key page layouts for the customer-facing website:

7.2.1 Home Page Structure

Hero Section	Full-width banner with search form overlay, background image
Featured Vehicles	Horizontal scroll or grid of 4-6 featured vehicles
Categories	Icon grid showing vehicle categories (SUV, Sedan, etc.)
Why Choose Us	Three-column benefit highlights with icons
Testimonials	Customer reviews carousel
CTA Section	Full-width call-to-action banner
Footer	Multi-column links, contact info, newsletter signup

7.2.2 Vehicle Listing Page Structure

Header	Search bar, category tabs
Filters Sidebar	Category, price range, transmission, features, dates
Results Header	Result count, sort dropdown, view toggle (grid/list)
Vehicle Grid	3-4 column responsive grid of vehicle cards
Pagination	Page numbers or "Load More" button

7.2.3 Booking Flow Structure

Step 1: Select Dates	Calendar picker, time selection, location options
Step 2: Your Details	Name, email, phone, license info form
Step 3: Review	Booking summary, price breakdown, T&C checkbox
Step 4: Confirmation	Success message, reference number, email confirmation

7.3 Admin Panel Wireframes

7.3.1 Dashboard Layout

Top Bar	Search, notifications bell, user menu
Sidebar	Navigation menu with icons, collapsible
KPI Row	4-5 key metrics cards (vehicles, rentals, revenue, etc.)
Charts Row	Revenue trend chart, status distribution pie
Activity Panel	Recent bookings, alerts, quick actions

7.3.2 Rental Management Layout

Header	Page title, "New Rental" button, status tabs
Filter Bar	Date range, status filter, search input
Data Table	Reference, customer, vehicle, dates, status, actions
Pagination	Rows per page selector, page navigation

7.3.3 Rental Detail Layout

Header	Reference number, status badge, action buttons
Info Cards	Customer info, vehicle info, dates/location
Status Timeline	Visual progression through rental stages
Inspection Tabs	Pickup inspection / Return inspection
Financial Summary	Costs, payments, outstanding balance
Notes Section	Customer notes, admin notes

7.4 Responsive Design

Breakpoints and layout adaptations:

Breakpoint	Width	Layout Changes
Mobile	< 640px	Single column, stacked elements, hamburger menu
Tablet	640px - 1024px	Two columns, collapsible sidebar, compact tables
Desktop	1024px - 1280px	Full layout, expanded sidebar, full tables
Large	> 1280px	Max-width container, extra whitespace

8. SECURITY DESIGN

8.1 Authentication & Authorization

8.1.1 Authentication Flow

The system uses Laravel Sanctum for API token authentication:

1. User submits email and password to /api/v1/auth/login
2. Server validates credentials against hashed password
3. On success, server generates a personal access token
4. Token is returned to client and stored securely
5. Client includes token in Authorization header for subsequent requests
6. Server validates token and retrieves associated user

8.1.2 Token Security

Security Measure	Implementation
Token Storage	HttpOnly cookies or secure local storage
Token Expiration	Configurable (default: 24 hours)
Token Refresh	Sliding expiration on activity
Token Revocation	Immediate logout, single or all devices
Rate Limiting	5 failed attempts triggers 15-min lockout

8.1.3 Role-Based Access Control (RBAC)

Permissions are managed through the Spatie Laravel Permission package:

Role	Permissions	Description
Super Admin	All permissions	Full system access, cannot be deleted
Admin	Most permissions	Manage vehicles, rentals, customers, reports
Manager	Limited permissions	Process rentals, view reports, manage assigned rentals
Staff	Basic permissions	View-only access, limited create capabilities

8.1.4 Permission Groups

Group	Permissions
Vehicles	vehicles.view, vehicles.create, vehicles.edit, vehicles.delete
Rentals	rentals.view, rentals.create, rentals.edit, rentals.process, rentals.complete
Customers	customers.view, customers.create, customers.edit, customers.blacklist

Reports	reports.view, reports.export
Settings	settings.view, settings.edit, users.manage, roles.manage

8.2 Data Protection

8.2.1 Encryption

Data Type	Encryption Method	Key Management
Passwords	bcrypt (cost 12)	N/A (one-way hash)
API Tokens	SHA-256 hash	Stored hashed in database
Sensitive Fields	AES-256-CBC	Laravel APP_KEY in .env
File Uploads	At-rest encryption (S3)	AWS KMS managed keys
Database Backups	GPG encryption	Separate backup key

8.2.2 Data at Rest

- Database: Encrypted tablespaces (MySQL) or full-disk encryption
- File Storage: S3 server-side encryption (SSE-S3 or SSE-KMS)
- Backups: Encrypted before upload to backup storage
- Logs: Sensitive data redacted, logs rotated and secured

8.2.3 Data in Transit

- TLS 1.2+ required for all connections
- HTTPS enforced via middleware and HSTS header
- API communications over HTTPS only
- Database connections use SSL/TLS

8.3 Input Validation

All user input is validated before processing:

```
// Laravel Form Request Example
class StoreRentalRequest extends FormRequest {
    public function rules(): array {
        return [
            "vehicle_id" => "required|exists:vehicles,id",
            "customer_id" => "required|exists:customers,id",
            "pickup_date" => "required|date|after:today",
            "return_date" => "required|date|after:pickup_date",
            "pickup_time" => "required|date_format:H:i",
            "deposit_amount" => "nullable|numeric|min:0",
        ];
    }
}
```

8.4 Security Headers

The following HTTP security headers are configured:

Header	Value	Purpose
Strict-Transport-Security	max-age=31536000; includeSubDomains	Enforce HTTPS

X-Content-Type-Options	nosniff	Prevent MIME sniffing
X-Frame-Options	SAMEORIGIN	Prevent clickjacking
X-XSS-Protection	1; mode=block	XSS filter
Content-Security-Policy	default-src self; ...	Control resource loading
Referrer-Policy	strict-origin-when-cross-origin	Control referrer info

9. INTEGRATION DESIGN

9.1 WhatsApp Business API Integration

9.1.1 Overview

WhatsApp Business API is used for instant notifications to admins and customers. The integration uses the official WhatsApp Cloud API.

9.1.2 Message Templates

Template Name	Trigger Event	Recipients
booking_confirmation	New booking created	Customer
booking_admin_alert	New booking created	Admin(s)
quote_request_alert	Quote request submitted	Admin(s)
return_reminder	24 hours before return	Customer
overdue_alert	Rental past return date	Admin(s) + Customer
rental_completed	Rental approved/completed	Customer

9.1.3 Implementation

```
class WhatsAppService {
    protected string $apiUrl;
    protected string $phoneNumberId;
    protected string $accessToken;

    public function sendTemplate(
        string $to,
        string $templateName,
        array $components = []
    ): WhatsAppResponse {
        $payload = [
            "messaging_product" => "whatsapp",
            "to" => $this->formatPhone($to),
            "type" => "template",
            "template" => [
                "name" => $templateName,
                "language" => ["code" => "en"],
                "components" => $components
            ]
        ];

        return $this->post("/messages", $payload);
    }
}
```

9.1.4 Error Handling

- Retry failed messages up to 3 times with exponential backoff
- Log all API responses for debugging
- Fallback to email if WhatsApp fails
- Queue messages for async processing

9.2 Email Service (SMTP)

9.2.1 Email Types

Email Type	Trigger	Template
Booking Confirmation	Successful booking	booking-confirmation.blade.php
Quote Request Received	Quote submission	quote-received.blade.php
Return Reminder	Scheduled job (24h before)	return-reminder.blade.php
Rental Summary	Rental completed	rental-summary.blade.php
Password Reset	User request	password-reset.blade.php

9.2.2 Configuration

```
// .env configuration
MAIL_MAILER=smtp
MAIL_HOST=smtp.gmail.com // or your SMTP provider
MAIL_PORT=587
MAIL_USERNAME=notifications@swiftflitz.com
MAIL_PASSWORD=app-specific-password
MAIL_ENCRYPTION=tls
MAIL_FROM_ADDRESS=no-reply@swiftflitz.com
MAIL_FROM_NAME="Swiftflitz Car Rental"
```

9.3 File Storage

9.3.1 Storage Configuration

The system supports both local and S3 storage with automatic switching:

Environment	Driver	Path/Bucket
Development	local	storage/app/public
Staging	local or s3	Configurable
Production	s3	swiftflitz-production-bucket

9.3.2 File Organization

```
storage/
├── vehicles/
│   ├── {vehicle_id}/
│   │   ├── images/
│   │   │   ├── main.jpg
│   │   │   └── gallery-{n}.jpg
│   │   └── documents/
│   └── rentals/
│       ├── {rental_id}/
│       │   ├── pickup/
│       │   │   ├── photos/
│       │   │   └── video.mp4
│       │   └── return/
│       │       └── photos/
│       └── customers/
│           ├── {customer_id}/
│           └── documents/
├── exports/
└── reports/
```


10. ERROR HANDLING & LOGGING

10.1 Error Handling Strategy

10.1.1 Exception Hierarchy

Custom exceptions provide specific error handling:

```
app/Exceptions/
├── Handler.php           // Global exception handler
├── BusinessException.php // Base business logic exception
├── VehicleNotAvailableException.php
├── RentalStatusException.php
├── CustomerBlacklistedException.php
├── PaymentException.php
└── NotificationException.php
```

10.1.2 HTTP Status Code Mapping

Exception Type	HTTP Status	User Message
ValidationException	422	Validation failed with specific field errors
AuthenticationException	401	Please log in to continue
AuthorizationException	403	You do not have permission for this action
ModelNotFoundException	404	Resource not found
BusinessException	422	Specific business rule message
ThrottleRequestsException	429	Too many requests, please try again later
Exception (other)	500	An unexpected error occurred

10.1.3 Error Response Structure

```
// Standard error response
{
  "success": false,
  "message": "Human-readable error message",
  "error_code": "VEHICLE_NOT_AVAILABLE", // Machine-readable code
  "errors": {                             // Validation errors only
    "field_name": ["Error message"]
  },
  "debug": {                               // Development only
    "exception": "VehicleNotAvailableException",
    "file": "VehicleService.php",
    "line": 125
  }
}
```

10.2 Logging Architecture

10.2.1 Log Channels

Channel	Driver	Purpose
stack	Multiple	Default channel (daily + slack)
daily	File	Application logs, rotated daily
slack	Slack webhook	Critical error alerts
audit	File	Security audit trail
api	File	API request/response logs

10.2.2 Log Levels

Level	Usage	Example
emergency	System unusable	Database connection failed
critical	Critical conditions	Payment gateway down
error	Error conditions	Failed to send notification
warning	Warning conditions	Deprecated API usage
notice	Normal but significant	User logged in from new device
info	Informational	Rental created successfully
debug	Debug information	SQL queries (dev only)

10.2.3 Audit Logging

All sensitive operations are logged for security audit:

- User authentication (login, logout, failed attempts)
- Permission changes
- Customer data modifications
- Financial transactions
- Data exports
- Configuration changes

11. PERFORMANCE DESIGN

11.1 Caching Strategy

11.1.1 Cache Layers

Layer	Technology	Cached Data	TTL
Application	Redis	Query results, computed values	1-60 minutes
Session	Redis	User sessions	2 hours
HTTP	Browser/CDN	Static assets	1 year
Database	MySQL query cache	Frequent queries	Auto-managed

11.1.2 Cache Keys

```
// Cache key patterns
"vehicles:list:{hash}"           // Paginated vehicle list
"vehicles:detail:{uuid}"        // Single vehicle
"vehicles:featured"             // Featured vehicles
"categories:all"                // All categories
"dashboard:stats:{date}"       // Dashboard KPIs
"user:{id}:permissions"        // User permissions
```

11.1.3 Cache Invalidation

Cache is invalidated on relevant data changes:

- Vehicle updated/deleted → Clear vehicle:detail:{uuid}, vehicles:list:*, vehicles:featured
- Rental status changed → Clear dashboard:stats:*
- Category modified → Clear categories:all, vehicles:list:*

11.2 Database Optimization

11.2.1 Query Optimization

- Eager loading to prevent N+1 queries
- Pagination for large result sets
- Selective column retrieval
- Proper indexing on frequently queried columns

11.2.2 Eager Loading Example

```
// Bad: N+1 query problem
$rentals = Rental::all();
foreach ($rentals as $rental) {
    echo $rental->vehicle->name; // Separate query each iteration
}

// Good: Eager loading
$rentals = Rental::with(["vehicle", "customer", "manager"]->get();
```

11.3 Asset Optimization

- JavaScript/CSS bundling and minification

- Image optimization (WebP format, responsive sizes)
- Lazy loading for images below the fold
- CDN for static assets in production
- Gzip/Brotli compression enabled

12. DEPLOYMENT ARCHITECTURE

12.1 Infrastructure Overview

The production environment runs on a VPS with the following components:

Component	Technology	Specification
Server	Ubuntu 22.04 LTS VPS	4 vCPU, 8GB RAM, 160GB SSD
Web Server	Nginx 1.24+	Reverse proxy, SSL termination
Application	PHP 8.2 + Laravel 11	PHP-FPM process manager
Database	MySQL 8.0	Same server or managed service
Cache	Redis 7	Session + application cache
Queue	Laravel Horizon + Redis	Background job processing
SSL	Let's Encrypt	Auto-renewal via Certbot

12.2 Environment Configuration

12.2.1 Environment Variables

Critical environment variables (stored in .env, never committed):

```
# Application
APP_NAME="Swiftflitz"
APP_ENV=production
APP_DEBUG=false
APP_URL=https://swiftflitz.com

# Database
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_DATABASE=swiftflitz_prod

# Cache & Queue
CACHE_DRIVER=redis
SESSION_DRIVER=redis
QUEUE_CONNECTION=redis

# External Services
WHATSAPP_PHONE_NUMBER_ID=xxxxxx
WHATSAPP_ACCESS_TOKEN=xxxxxx
```

12.3 CI/CD Pipeline

Deployment pipeline using GitHub Actions:

- Developer pushes to main branch
- GitHub Actions triggers workflow
- Run automated tests (PHPUnit, Jest)

10. Build frontend assets (npm run build)
11. Deploy to staging (if tests pass)
12. Manual approval for production
13. Deploy to production via SSH
14. Run migrations, clear caches
15. Restart queue workers
16. Notify team via Slack

13. APPENDICES

Appendix A: Glossary

Term	Definition
Rental	A booking where a customer takes a vehicle for a period
Inspection	Documentation of vehicle condition at pickup or return
Quote Request	Customer inquiry for vehicles without listed prices
Blacklist	Status preventing customer from making bookings
Featured Vehicle	Vehicle highlighted on the homepage
Overdue	Rental where return date has passed without vehicle return

Appendix B: Database Schema SQL

Complete database migration scripts are available in the Laravel project under database/migrations/. Key tables include:

- users, roles, permissions, role_has_permissions
- vehicles, vehicle_images, categories
- customers, customer_documents
- rentals, rental_inspections
- quote_requests
- notifications, notification_logs
- settings

END OF DOCUMENT

ORDAQ-SF-SDD-2026-001 | Version 1.0 | January 2026

Prepared by Ordaq Tech Solutions